Index.html

Description: The index html file allows users to browse menu items via the search bar with features for filtering shown items by allergens and dietary restrictions. This file includes navigation links to the other pages and uses the sidebar to dynamically load menu items based on the user's responses to the filter options.

```
<body>
  <header>
    <!--Header search to allow users to search for menu items-->
    <div class="header-search">
      <!--Input box for keyword search-->
      <input type="text" id="searchBox" placeholder="Search menu items"/>
      <!--Button triggers the search in app.js-->
      <button id="searchBtn">Search</button>
    </div>
    <!--Navigation links for swapping pages-->
    <nav class="header-nav">
      <a href="index.html">Menu</a>
      <a href="cart.html">Cart</a>
      <a href="orders.html">Orders</a>
    </nav>
  </header>

  <main class="page">
    <!--side bar for filtering items on menu-->
    <aside class="filters">
      <h2>Filters</h2>
      <!--Food Filter-->
      <label for="categoryFilter">Category</label>
      <select id="categoryFilter">
        <option value="All">All</option>
        <option value="Burgers">Burgers</option>
        <option value="Wraps">Wraps</option>
        <option value="Salads">Salads</option>
      </select>
      <!--Allergen Filter-->
      <h3>Exclude Allergens</h3>
      <div class="checkbox-group">
        <label><input type="checkbox" value="Dairy" class="allergenCheckbox">Dairy</label>
        <label><input type="checkbox" value="Gluten" class="allergenCheckbox">Gluten</label>
        <label><input type="checkbox" value="Soy" class="allergenCheckbox">Soy</label>
        <label><input type="checkbox" value="Nuts" class="allergenCheckbox">Nuts</label>
      </div>
      <!--Dietary Restrictions Filter-->
      <h3>Dietary Restrictions</h3>
      <div class="checkbox-group">
```

```
            <label><input type="checkbox" value="Vegetarian"
class="dietaryCheckbox">Vegetarian</label>
            <label><input type="checkbox" value="Vegan" class="dietaryCheckbox">Vegan</label>
            <label><input type="checkbox" value="Gluten Free" class="dietaryCheckbox">Gluten
Free</label>
        </div>
        <!--Applies filter by calling js function-->
        <button id="applyFilterButton">Apply Filters</button>
        <!--resets applied filters and reloads page-->
        <button id="resetFiltersButton" class="secondary-button">Reset</button>
    </aside>

    <section class="content">
        <h1>Menu</h1>
        <div id="menuContainer"></div>
    </section>
  </main>

  <script src="script.js"></script>
</body>
</html>
```

Cart.html

Description: This file is where all the menu items are stored for the customers “shopping cart” to view the items they have selected. It also includes a breakdown of the order total, tax value, delivery fees and optional tip value.

```
  <body>
  <main class="page">
    <section class="content">
        <h1>Your Cart</h1>
        <!--container where items are dynamically rendered-->
        <div id="cartContainer">
            <p>Your cart items are located here</p>
        </div>
        <!--displays calculated totals-->
        <div id="cartTotals">
            <p><strong>Subtotal:</strong> <span id="subtotalValue">$0.00</span></p>
            <p><strong>Tax:</strong> <span id="taxValue">$0.00</span></p>
            <p><strong>Delivery Fee:</strong> <span id="deliveryFeeValue">$0.00</span></p>
            <p><strong>Service Fee:</strong> <span id="serviceFeeValue">$0.00</span></p>
            <p><strong>Tip:</strong> <span id="tipValue">$0.00</span></p>
            <h3>Total: <span id="totalValue">$0.00</span></h3>
        </div>

        <!--Navigation buttons-->
        <div class="item-actions">
            <a href="index.html">Continue Shopping</a>
```

```
            <a href="checkout.html">Proceed to Checkout</a>
          </div>
        </section>
    </main>
    <!--loads cart functionality  -->
    <script src="cart.js"></script>
</body>
</html>
```

Checkout.html

Description: This file handles the process for checking out a user's cart. Users will be able to enter delivery details, payment information and the option to tip the service worker. After the checkout process this file will present the customer with a confirmation message.

```
<body>
    <main class="page">
        <section class="content">
            <h1>Checkout</h1>
            <!--form to collect order info and customer info-->
            <form id="checkoutForm">
                <!--pickup or delivery selection-->
                <label for="orderType">Order Type</label>
                <select id="orderType" required>
                    <option value="Delivery">Delivery</option>
                    <option value="Pickup">Pickup</option>
                </select>
                <!--delivery address required if delivery is selected-->
                <div id="deliveryAddressGroup">
                    <label for="deliveryAddress">Delivery Address</label>
                    <input type="text" id="deliveryAddress" />
                </div>
                <!--customer info and payment info-->
                <label for="contactEmail">Email</label>
                <input type="email" id="contactEmail" required />
                <label for="cardholderName">Cardholder Name</label>
                <input type="text" id="cardholderName" required />
                <label for="cardNumber">Card Number</label>
                <input type="text" id="cardNumber" required />
                <!--tip option-->
                <label for="tipAmount">Tip</label>
                <input type="number" id="tipAmount" min="0" step="0.01" value="0" />
                <!--places order-->
                <button type="submit">Place Order</button>
            </form>
            <!--order confirmation success/failure message-->
           <p id="checkoutConfirmation"></p>
        </section>
    </main>
```

```
    <script src="checkout.js"></script>
</body>
</html>
```

Orders.html

Description: This file displays the users previous order history and shows the contents of the previously placed orders. This content is dynamically generated using js to allow users to view their previous purchases and reorder.

```
<body>
    <main class="page">
        <section class="content">
            <h1>Order History</h1>
            <!--js loads past orders-->
            <div id="ordersContainer">
                <p>Your previous orders are shown here.</p>
            </div>
        </section>
    </main>

    <script src="orders.js"></script>
</body>
</html>
```

Confirmation.html

Description: This file displays after a successful order is placed and confirms to the user that the process and transaction was completed. This file also displays the generated order Id associated with the customer's order.

```
    <main class="page">
        <section class="content">
            <!--confirmation message-->
            <h1>Order Confirmed</h1>
            <p>Your order has been placed successfully.</p>
            <!--js inserts the order id-->
            <p id="orderIdMessage">Order ID</p>
            <!--navigation options -->
            <div class="item-actions">
                <a href="index.html">Back to Menu</a>
                <a href="orders.html">View Orders</a>
            </div>

        </section>
    </main>
    <!--displays order id and confirmation-->
    <script src="confirmation.js"></script>
</body>
```

```
</html>
```

Checkout.js

Des: This file handles the checkout process. It validates the form fields, checks if the cart has items, calculates subtotal, tax, delivery fee, service fee, and tip, then it sends the order data to the backend server. After the order is successfully saved, it redirects the user to the confirmation page.

```
const form = document.getElementById("checkoutForm");
const orderType = document.getElementById("orderType");
const address = document.getElementById("deliveryAddress");
const email = document.getElementById("contactEmail");
const name = document.getElementById("cardholderName");
const card = document.getElementById("cardNumber");
const tipInput = document.getElementById("tipAmount");
const message = document.getElementById("checkoutConfirmation");

// gets cart from local storage
function getCart() {
    return JSON.parse(localStorage.getItem("cart")) || [];
}

// checks if card is 16 digits
function validCard(num) {
    return /^\d{16}$/.test(num);
}

// shows or hides address if pickup or delivery changes
function toggleAddress() {
    if (!orderType || !address) {
        return;
    }

    const addressGroup = document.getElementById("deliveryAddressGroup");

    if (!addressGroup) {
        return;
    }

    if (orderType.value === "Delivery") {
        addressGroup.style.display = "block";
        address.required = true;
    } else {
```

```
        addressGroup.style.display = "none";
        address.required = false;
        address.value = "";
    }
}

if (orderType) {
    orderType.addEventListener("change", toggleAddress);
    toggleAddress();
}

if (form) {
    form.addEventListener("submit", async function (e) {
        e.preventDefault();

        const cart = getCart();

        if (cart.length === 0) {
            message.textContent = "Cart is empty";
            return;
        }

        if (orderType.value === "Delivery" && address.value.trim() === "") {
            message.textContent = "Enter delivery address";
            return;
        }

        if (email.value.trim() === "") {
            message.textContent = "Enter email";
            return;
        }

        if (name.value.trim() === "") {
            message.textContent = "Enter name";
            return;
        }

        if (!validCard(card.value.trim())) {
            message.textContent = "Invalid card number";
            return;
        }

        let subtotal = 0;

        cart.forEach(function (item) {
            subtotal += item.price * item.quantity;
        });

        const tax = subtotal * 0.07;
```

```
        const delivery = orderType.value === "Delivery" ? 3.99 : 0;
        const service = 1.99;
        const tip = Number(tipInput.value) || 0;
        const total = subtotal + tax + delivery + service + tip;

        // sends order to backend
        const orderData = {
            orderType: orderType.value,
            address: address.value.trim(),
            email: email.value.trim(),
            cardholderName: name.value.trim(),
            subtotal: subtotal,
            tax: tax,
            deliveryFee: delivery,
            serviceFee: service,
            tip: tip,
            total: total,
            items: cart
        };

        try {
            const response = await fetch("http://localhost:3000/api/orders", {
                method: "POST",
                headers: {
                    "Content-Type": "application/json"
                },
                body: JSON.stringify(orderData)
            });

            const data = await response.json();

            if (!response.ok) {
                message.textContent = "Order failed";
                return;
            }

            localStorage.removeItem("cart");
            window.location.href = `confirmation.html?orderId=${data.orderId}`;
        } catch (error) {
            message.textContent = "Server error";
        }
    });
}
```

Orders.js

Des: This file loads and displays all previously placed orders from the backend server. If there are no orders, it shows a message saying there are no orders yet. Otherwise, it shows the order ID, date, order type, and total.

```
const container = document.getElementById("ordersContainer");

// loads orders from backend
async function loadOrders() {
    if (!container) {
        return;
    }

    try {
        const response = await fetch("http://localhost:3000/api/orders");
        const orders = await response.json();

        container.innerHTML = "";

        if (orders.length === 0) {
            container.innerHTML = "<p>No orders yet</p>";
            return;
        }

        orders.forEach(function (order) {
            const div = document.createElement("div");
            div.className = "order-card";

            div.innerHTML = `
                <h3>Order ID: ${order.id}</h3>
                <p>Date: ${order.created_at}</p>
                <p>Type: ${order.order_type}</p>
                <p>Total: $${Number(order.total).toFixed(2)}</p>
            `;

            container.appendChild(div);
        });
    } catch (error) {
        container.innerHTML = "<p>Error loading orders</p>";
    }
}

loadOrders();
```

Confirmation.js

Des: This file displays the confirmation information after checkout. It gets the order ID from the URL, requests the order from the backend server, and shows the order ID on the confirmation page. If no order is found, it displays a fallback message.

```
const message = document.getElementById("orderIdMessage");

// gets order id from url
function getOrderId() {
    const params = new URLSearchParams(window.location.search);
    return params.get("orderId");
}

// shows order from backend
async function showOrder() {
    const orderId = getOrderId();

    if (!orderId) {
        message.textContent = "No order found";
        return;
    }

    try {
        const response = await fetch(`http://localhost:3000/api/orders/${orderId}`);
        const order = await response.json();

        if (!order) {
            message.textContent = "Order not found";
            return;
        }

        message.textContent = "Order ID: " + order.id;
    } catch (error) {
        message.textContent = "Error loading order";
    }
}

showOrder();
```

Cart.js

Des: This file handles the shopping cart state. It dynamically retrieves items from localStorage, populates the cart items in the HTML, calculates standard totals (subtotal, tax, service fees, tips, delivery fees), and allows the user to increase/decrease item quantities or remove them completely updating the localStorage in real-time.

```
document.addEventListener("DOMContentLoaded", function () {
    const cartContainer = document.getElementById("cartContainer");
    const subtotalValue = document.getElementById("subtotalValue");
    const taxValue = document.getElementById("taxValue");
    const deliveryFeeValue = document.getElementById("deliveryFeeValue");
    const serviceFeeValue = document.getElementById("serviceFeeValue");
    const tipValue = document.getElementById("tipValue");
    const totalValue = document.getElementById("totalValue");

    function renderCart() {
        const cart = JSON.parse(localStorage.getItem("cart")) || [];

        cartContainer.innerHTML = ""; // Clear existing items

        if (cart.length === 0) {
            cartContainer.innerHTML = "<p>Your cart is currently empty.</p>";
            updateTotals(0);
            return;
        }

        let subtotal = 0;

        cart.forEach((item, index) => {
            const itemTotal = item.price * item.quantity;
            subtotal += itemTotal;

            const itemDiv = document.createElement("div");
            itemDiv.className = "cart-item";
            itemDiv.innerHTML = `
                <div class="cart-item-details">
                    <h4>${item.name}</h4>
                    <p>Price: $${Number(item.price).toFixed(2)}</p>
                    <p>
                        Quantity:
                        <button onclick="changeQuantity(${index}, -1)">-</button>
                        ${item.quantity}
                        <button onclick="changeQuantity(${index}, 1)">+</button>
                    </p>
                    <p>Total: $${itemTotal.toFixed(2)}</p>
```

```
            <button onclick="removeItem(${index})">Remove</button>
        </div>
        <hr>
    `;
    cartContainer.appendChild(itemDiv);
  });

  updateTotals(subtotal);
}

function updateTotals(subtotal) {
  const taxRate = 0.07; // 7% tax
  const serviceRate = 0.05; // 5% service fee
  const deliveryFee = subtotal > 0 ? 3.99 : 0; // example $3.99 delivery fee
  const tip = 0; // standard $0 tip for now

  const tax = subtotal * taxRate;
  const serviceFee = subtotal * serviceRate;
  const finalTotal = subtotal + tax + deliveryFee + serviceFee + tip;

  subtotalValue.textContent = `$${subtotal.toFixed(2)}`;
  taxValue.textContent = `$${tax.toFixed(2)}`;
  deliveryFeeValue.textContent = `$${deliveryFee.toFixed(2)}`;
  serviceFeeValue.textContent = `$${serviceFee.toFixed(2)}`;
  tipValue.textContent = `$${tip.toFixed(2)}`;
  totalValue.textContent = `$${finalTotal.toFixed(2)}`;

  // Save the summary to localStorage in case we need it at checkout
  localStorage.setItem("cartSummary", JSON.stringify({
    subtotal, tax, deliveryFee, serviceFee, tip, total: finalTotal
  }));
}

// Expose functions globally for the onclick handlers
window.changeQuantity = function (index, delta) {
  let cart = JSON.parse(localStorage.getItem("cart")) || [];
  if (cart[index]) {
    cart[index].quantity += delta;

    // Remove item if quantity falls to 0 or below
    if (cart[index].quantity <= 0) {
      cart.splice(index, 1);
    }

    localStorage.setItem("cart", JSON.stringify(cart));
    renderCart(); // Re-render
  }
};
```

```
    window.removeItem = function (index) {
        let cart = JSON.parse(localStorage.getItem("cart")) || [];
        cart.splice(index, 1);
        localStorage.setItem("cart", JSON.stringify(cart));
        renderCart(); // Re-render
    };

    // Initial render
    renderCart();
});
```

Script.js

Des: This file handles the interactive logic for the homepage menu. It fetches the menu items from the Node.js backend to display dynamically, drives the multi-layered filter and search logic, and processes interactions to save selected items to localStorage via addToCart().

```
let allMenuItems = [];

// Wait for DOM to load
document.addEventListener("DOMContentLoaded", function () {
    const menuContainer = document.getElementById("menuContainer");
    const applyFilterButton = document.getElementById("applyFilterButton");
    const resetFiltersButton = document.getElementById("resetFiltersButton");
    const searchBtn = document.getElementById("searchBtn");

    // fetch menu items from backend
    if (menuContainer) {
        fetch("http://localhost:3000/api/menu")
            .then(response => response.json())
            .then(items => {
                allMenuItems = items; // Store the original fetched data
                renderMenu(allMenuItems);
            })
            .catch(err => {
                console.error(err);
                menuContainer.innerHTML = "<p>Server error loading menu</p>";
            });
    }

    // Function to render items dynamically
    function renderMenu(items) {
        if (!menuContainer) return;
        menuContainer.innerHTML = ""; // clear container
```

```
        if (items.length === 0) {
            menuContainer.innerHTML = "<p>No menu items available matching your criteria.</p>";
            return;
        }

        // dynamically generate HTML for each menu item
        items.forEach(item => {
            const div = document.createElement("div");
            div.className = "menu-item-card";

            div.innerHTML = `
                <h2>${item.name} - $${Number(item.price).toFixed(2)}</h2>
                <p><strong>Category:</strong> ${item.category}</p>
                <p>${item.description}</p>
                <p><em>Allergens:</em> ${item.allergens}</p>
                <p><em>Dietary Tags:</em> ${item.dietary_tags}</p>
                <button onclick="addToCart(${item.id}, '${item.name}', ${item.price})" ${item.in_stock ? '' :
'disabled'}>
                    ${item.in_stock ? 'Add to Cart' : 'Out of Stock'}
                </button>
            `;

            menuContainer.appendChild(div);
        });
    }

    // Apply Filters logic
    if (applyFilterButton) {
        applyFilterButton.addEventListener("click", () => {
            const categorySelect = document.getElementById("categoryFilter").value;
            const excludeAllergens =
Array.from(document.querySelectorAll(".allergenCheckbox:checked")).map(cb =>
cb.value.toLowerCase());
            const requiredDietary =
Array.from(document.querySelectorAll(".dietaryCheckbox:checked")).map(cb =>
cb.value.toLowerCase());
            const searchQuery = document.getElementById("searchBox") ?
document.getElementById("searchBox").value.toLowerCase() : "";

            const filteredItems = allMenuItems.filter(item => {
                // 1. Search Query Box
                if (searchQuery && !item.name.toLowerCase().includes(searchQuery) &&
!item.description.toLowerCase().includes(searchQuery)) {
                    return false;
                }

                // 2. Category filter
                if (categorySelect !== "All" && item.category !== categorySelect) {
```

```
                return false;
            }

            // 3. Allergen filter (EXCLUDE if item HAS any of the checked allergens)
            let itemAllergens = (item.allergens || "").toLowerCase();
            for (let allergen of excludeAllergens) {
                if (itemAllergens.includes(allergen)) {
                    return false; // Item has this allergen, exclude it
                }
            }

            // 4. Dietary filter (INCLUDE only if item HAS ALL of the checked dietary tags)
            let itemDietary = (item.dietary_tags || "").toLowerCase();
            for (let diet of requiredDietary) {
                if (!itemDietary.includes(diet)) {
                    return false; // Item lacks this required dietary tag, exclude it
                }
            }

            return true; // Passed all filters!
        });

        renderMenu(filteredItems);
    });
}

// Reset Filters logic
if (resetFiltersButton) {
    resetFiltersButton.addEventListener("click", () => {
        // Reset Dropdown
        const categorySelect = document.getElementById("categoryFilter");
        if (categorySelect) categorySelect.value = "All";

        // Reset Checkboxes
        document.querySelectorAll(".allergenCheckbox, .dietaryCheckbox").forEach(cb => cb.checked =
false);

        // Reset Search Box
        const searchBox = document.getElementById("searchBox");
        if (searchBox) searchBox.value = "";

        // Re-render everything
        renderMenu(allMenuItems);
    });
}

// Search Button logic (triggers the same filter)
if (searchBtn) {
    searchBtn.addEventListener("click", () => {
```

```
        if (applyFilterButton) applyFilterButton.click();
    });
  }
});

// universal function to add to cart stored in localStorage
function addToCart(id, name, price) {
  const cart = JSON.parse(localStorage.getItem("cart")) || [];

  // check if it already exists to update quantity
  const existingIndex = cart.findIndex(item => item.id === id);
  if (existingIndex > -1) {
    cart[existingIndex].quantity += 1;
  } else {
    cart.push({ id, name, price, quantity: 1 });
  }

  localStorage.setItem("cart", JSON.stringify(cart));
  alert(name + " added to cart!");
}
```

Server.js

Des: This serves as the Express REST API backend entry point. It creates an explicit filesystem connection to execute database.sql seeding instructions into SQLite across startup, dynamically routes fetch requests between the API (/api/menu, /api/orders) and SQL queries, and statically hosts the static html files.

```
const express = require("express");
const cors = require("cors");
const sqlite3 = require("sqlite3").verbose();
const path = require("path");

const app = express();
const PORT = 3000;

// middleware
app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, "../frontend")));

// connect to database
const fs = require("fs");
const db = new sqlite3.Database(path.join(__dirname, "orders.db"), function (err) {
```

```
    if (err) {
        console.error("Database connection failed:", err.message);
    } else {
        console.log("Connected to database");

        // initialize database schema and seed data from database.sql
        const initScript = fs.readFileSync(path.join(__dirname, "database.sql"), "utf-8");
        db.exec(initScript, function (err) {
            if (err) {
                console.error("Failed to execute database.sql:", err.message);
            } else {
                console.log("Database tables verified and seeded.");
            }
        });
    }
});

// get all menu items
app.get("/api/menu", function (req, res) {
    const sql = `SELECT * FROM menu_items`;

    db.all(sql, [], function (err, rows) {
        if (err) {
            console.error(err.message);
            return res.status(500).json({ message: "Could not load menu" });
        }
        res.json(rows);
    });
});

// create new order
app.post("/api/orders", function (req, res) {
    const {
        orderType,
        address,
        email,
        cardholderName,
        subtotal,
        tax,
        deliveryFee,
        serviceFee,
        tip,
        total,
        items
    } = req.body;

    if (!orderType || !email || !cardholderName || !items || items.length === 0) {
        return res.status(400).json({ message: "Missing required fields" });
    }
```

```
    const sql = `
        INSERT INTO orders (
            order_type,
            address,
            email,
            cardholder_name,
            subtotal,
            tax,
            delivery_fee,
            service_fee,
            tip,
            total,
            items
        )
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    `;

    const values = [
        orderType,
        address || "",
        email,
        cardholderName,
        subtotal,
        tax,
        deliveryFee,
        serviceFee,
        tip,
        total,
        JSON.stringify(items)
    ];

    db.run(sql, values, function (err) {
        if (err) {
            console.error(err.message);
            return res.status(500).json({ message: "Could not save order" });
        }

        res.status(201).json({
            message: "Order saved",
            orderId: this.lastID
        });
    });
});

// get all orders
app.get("/api/orders", function (req, res) {
    const sql = `SELECT * FROM orders ORDER BY created_at DESC`;
```

```
    db.all(sql, [], function (err, rows) {
        if (err) {
            console.error(err.message);
            return res.status(500).json({ message: "Could not load orders" });
        }

        res.json(rows);
    });
});

// get one order by id
app.get("/api/orders/:id", function (req, res) {
    const sql = `SELECT * FROM orders WHERE id = ?`;

    db.get(sql, [req.params.id], function (err, row) {
        if (err) {
            console.error(err.message);
            return res.status(500).json({ message: "Could not load order" });
        }

        if (!row) {
            return res.status(404).json({ message: "Order not found" });
        }

        res.json(row);
    });
});

// start server
app.listen(PORT, function () {
    console.log(`Server running on http://localhost:${PORT}`);
});
```